

Building an AI Observability Platform with OpenTelemetry and Phoenix

After building a source-grounded RAG assistant, the next architectural question is not only whether the assistant can answer questions.

The next question is whether the platform can explain what happened inside the AI workflow.

A RAG system may retrieve documents, construct prompts, call a model, return an answer, and cite sources. But without observability, the platform cannot reliably answer the operational questions that matter:

Was retrieval slow?

Did the model call take too long?

How many tokens were used?

Did retrieval find useful context?

Was the answer supported by sources?

Did the request technically succeed but produce a weak result?

Traditional API monitoring is not enough for this kind of system. A `200 OK` response only tells us that the endpoint returned successfully. It does not tell us whether the AI workflow behaved well.

That is the purpose of the `05-ai-observability-platform` project.

It builds an instrumented AI API using FastAPI, OpenTelemetry traces, Arize Phoenix integration, local metrics storage, and a lightweight dashboard. The goal is to make AI behavior measurable, inspectable, and easier to operate.

From an AI Data Platform Architect's perspective, this project represents the observability layer that every serious AI platform eventually needs.

Why AI Observability Matters

AI applications behave differently from traditional APIs.

A normal backend request often follows a relatively clear path:

Receive input.

Validate the request.

Call a database or service.

Return a response.

An AI request usually has more moving parts.

It may retrieve context, build a prompt, call a model, count tokens, evaluate grounding, score quality, and handle generation-specific failures. Each of those stages can affect the final user experience.

A request can succeed from an HTTP perspective and still fail from an AI quality perspective.

For example:

The endpoint may return `200 OK`, but retrieval may have found weak context.

The model may return a fluent answer, but the answer may not be grounded.

The API may be available, but model latency may be too high.

The request may complete, but token usage may spike unexpectedly.

The system may avoid exceptions, but answer quality may degrade over time.

That is why AI observability needs to measure more than uptime, latency, and error rate.

Those traditional metrics still matter, but they are incomplete.

AI platforms need visibility into:

- model latency
- retrieval latency
- prompt token count
- completion token count
- total token usage
- retrieval hit rate
- source count
- quality score
- model or generation errors
- trace context across the full AI workflow

This project treats those signals as first-class platform metrics.

Project Overview

The project implements a small AI API that simulates an observable RAG-style workflow.

The API exposes one primary instrumented endpoint:

```
POST /ask
```

When a request arrives, the system runs an observed AI pipeline:

```
POST /ask
-> ai.request
    -> ai.retrieval
    -> ai.model.generate
    -> ai.quality.evaluate
```

Each stage records timing and AI-specific attributes.

The application also stores request-level metrics in a local SQLite database and exposes metrics through API endpoints and a local dashboard.

The stack includes:

Layer	Technology
API	FastAPI + Uvicorn
Tracing	OpenTelemetry
Trace viewer	Arize Phoenix via OTLP
Local metrics	SQLite
Dashboard	Built-in HTML dashboard + dashboard spec
Reliability docs	Markdown report

The architecture is intentionally lightweight, but the pattern maps directly to production AI systems.

The core idea is simple: observe the AI request as a workflow, not as a single opaque API call.

The Instrumented AI Workflow

The core of the project is an observed `/ask` workflow.

At a high level, the request performs four steps:

1. Receives a user question.
2. Retrieves relevant context from a small local knowledge base.
3. Generates a mock answer from the retrieved context.
4. Scores a simple quality signal based on source support.

The mock generation is not the most important part.

The important part is how the system measures each stage.

The API records:

- total request latency
- retrieval latency
- model generation latency
- prompt tokens
- completion tokens
- total tokens
- retrieval hit status
- source count
- quality score
- error status
- exception details

This creates a measurement framework that can later be connected to real retrieval, real model serving, and real quality evaluation.

That is the platform value.

The project does not only demonstrate an AI endpoint. It demonstrates how to instrument the endpoint so the platform can understand its behavior.

OpenTelemetry Trace Design

OpenTelemetry gives the platform a way to represent an AI request as a trace.

Instead of treating `/ask` as one black-box request, the project breaks the workflow into spans.

The span map is:

Span	Purpose
<code>POST /ask</code>	FastAPI request span
<code>ai.request</code>	Root span for the AI workflow

Span	Purpose
<code>ai.retrieval</code>	Measures retrieval work
<code>ai.model.generate</code>	Measures model generation work
<code>ai.quality.evaluate</code>	Measures response quality evaluation

This structure matters because AI latency is rarely one thing.

If a user says the assistant is slow, the platform needs to isolate where the time was spent.

Was the API slow?

Was retrieval slow?

Was model generation slow?

Did the prompt get too large?

Did quality evaluation add overhead?

Did an exception occur during retrieval or generation?

A single request latency number cannot answer those questions.

A trace can.

By splitting the workflow into spans, the platform can inspect each part of the request independently.

This is one of the most important differences between basic API monitoring and AI workflow observability.

AI-Specific Trace Attributes

The project adds AI-specific attributes to the OpenTelemetry spans.

These include:

- `ai.question.length`
- `ai.model.name`
- `ai.request.latency_ms`
- `ai.retrieval.latency_ms`
- `ai.model.latency_ms`
- `ai.tokens.prompt`
- `ai.tokens.completion`

- `ai.tokens.total`
- `ai.retrieval.hit`
- `ai.quality.score`

These attributes turn traces into operational evidence.

A trace should not only show that a model-generation span occurred. It should show which model was used, how long it took, how many tokens were involved, and whether the overall request quality looked acceptable.

This is especially important when debugging AI systems.

If model latency increases, the trace can show whether the prompt tokens increased.

If quality drops, the trace can show whether retrieval returned any sources.

If total latency spikes, the trace can show whether the bottleneck was retrieval, generation, or quality evaluation.

If errors occur, the trace can show exactly where they happened.

From an AI Data Platform Architect's perspective, these attributes are the beginning of an AI control plane. They connect system behavior, retrieval behavior, and model behavior in one observable workflow.

Phoenix Integration

The project supports exporting OpenTelemetry traces to Arize Phoenix using OTLP.

The local flow is:

```
FastAPI app
-> OpenTelemetry spans
-> OTLP gRPC exporter
-> Phoenix collector on :4317
-> Phoenix UI on :6006
```

The Docker Compose setup runs the API and Phoenix together.

In that mode, the API exports traces to:

```
http://phoenix:4317
```

Phoenix provides a visual trace waterfall.

That visual view is useful because many AI issues are easier to understand when the workflow is displayed as a timeline.

A slow request may show a long `ai.model.generate` span.

A weak response may show an `ai.retrieval` span with no hit.

A failed request may show the exact span where an exception occurred.

A token spike may correlate with longer model-generation latency.

This type of trace inspection is essential for AI platform debugging.

The project also supports a console exporter for simpler local testing:

```
OTEL_EXPORTER=console uvicorn api.app.main:app --reload --port 8003
```

That makes it possible to inspect raw spans without running the full Phoenix stack.

This is a useful local development pattern: support lightweight console inspection first, then export traces to a dedicated UI when deeper analysis is needed.

Local Metrics Store

Traces are excellent for investigating individual requests.

But platform teams also need aggregate metrics.

A single trace can explain one request. Metrics explain system behavior over time.

This project stores request-level metrics in SQLite. Each `/ask` request records:

- timestamp
- route
- status
- model name
- request latency
- retrieval latency
- model latency
- prompt tokens
- completion tokens
- total tokens
- retrieval hit

- quality score
- error message

The API exposes two metrics endpoints:

```
GET /metrics/summary
GET /metrics/events
```

`/metrics/summary` returns aggregate reliability metrics such as:

- total requests
- error count
- error rate
- average request latency
- average retrieval latency
- average model latency
- average total tokens
- retrieval hit rate
- average quality score

`/metrics/events` returns recent request-level events.

This design creates two useful levels of visibility.

The summary endpoint shows the overall health of the system.

The events endpoint shows recent request details.

In a production platform, SQLite would likely be replaced with Prometheus, OTLP metrics, a warehouse-backed events table, or a managed observability tool. But the local SQLite implementation is still valuable because it defines the shape of the telemetry data.

The important architectural point is not the storage engine.

The important point is deciding which AI signals should be captured.

Dashboard Design

The project includes a lightweight local dashboard at:

```
http://localhost:8003/dashboard
```


The dashboard summarizes:

- total requests
- error rate
- average request latency
- average retrieval latency
- average model latency
- average token usage
- retrieval hit rate
- average quality score
- recent request events

The project also includes a dashboard specification file:

```
dashboards/ai-observability-dashboard.json
```

That dashboard spec defines panels for:

Panel	Why It Matters
Request latency	User-facing performance
Model latency	LLM runtime bottlenecks
Retrieval latency	Vector store or retrieval bottlenecks
Token usage	Cost and context pressure
Error rate	Reliability baseline
Retrieval hit rate	RAG grounding signal
Quality score	Response usefulness and source support

From an architecture perspective, the dashboard is not just a visualization. It is an operating model.

A dashboard defines what the platform team believes is important enough to monitor.

For AI systems, that operating model needs to include both infrastructure health and AI behavior.

A traditional dashboard might show request count, latency, and error rate.

An AI observability dashboard needs to go further. It needs to show whether the AI workflow is retrieving context, staying within token expectations, generating responses in a reasonable time, and producing quality signals that are trending in the right direction.

Reliability Goals

The reliability report defines several goals for the AI workflow:

Goal	Signal
Fast enough for users	Request latency and model latency stay within target
Grounded answers	Retrieval hit rate and source count remain high
Predictable model usage	Token counts do not spike unexpectedly
Clear failures	Errors are captured with trace context
Measurable quality	Quality score trends are visible over time

These goals reflect a broader truth: AI reliability is not only uptime.

An AI system can be available but slow.

It can be fast but ungrounded.

It can return responses but use too many tokens.

It can avoid errors but produce low-quality answers.

It can retrieve context but fail to use it well.

It can generate a fluent response that is not supported by the source material.

Observability needs to account for all of those scenarios.

That is why this project treats retrieval hit rate, token usage, quality score, and trace context as reliability signals.

Suggested SLOs

The project proposes initial service-level objectives for the local lab.

These are not production commitments. They are local targets that demonstrate the practice of defining measurable expectations.

SLO	Initial Target
<code>/ask</code> availability	99% successful responses locally during test runs

SLO	Initial Target
p95 request latency	Under 2 seconds for mock pipeline
p95 retrieval latency	Under 250 ms for local retrieval
p95 model latency	Under 1.5 seconds for local model calls
Retrieval hit rate	Above 90% on known-answer questions
Error rate	Below 1% during steady-state runs

The important point is not the exact numbers.

The important point is that AI platforms need explicit expectations.

Without targets, teams rely on anecdotal feedback.

The assistant feels slow.

The answers seem worse.

The prompt might be too large.

The model appears unreliable.

With SLOs, the platform can define what good looks like and measure whether the system is meeting that standard.

This is how AI systems move from demos to operable services.

AI Failure Modes

The reliability report identifies several important AI failure modes:

Failure	Detection Signal	Response
Model runtime slow	High <code>ai.model.latency_ms</code>	Reduce model size, enable streaming, scale serving
Retrieval degraded	Low retrieval hit rate	Reindex documents, tune chunking, improve embeddings
Prompt too large	Token count spike	Add context compression, lower retrieved chunk count
Model exception	Error span and error metric	Retry safe failures, surface fallback response

Failure	Detection Signal	Response
Poor answer quality	Quality score drops	Run eval set, inspect prompt/context, add guardrails

This is where AI observability becomes different from traditional API observability.

A platform team needs to know not only whether the service is up, but whether the AI workflow is behaving responsibly.

Retrieval can degrade.

Token usage can become expensive.

Prompts can grow too large.

Quality can drop silently.

Model calls can fail in ways that require trace context.

The failure modes are not just infrastructure failures. They are AI system failures.

A good observability layer needs to detect both.

Why This Architecture Matters

AI data platforms sit at the intersection of data systems, model systems, and application systems.

They need to support:

- retrieval-augmented generation
- embedding workflows
- model-serving APIs
- evaluation pipelines
- prompt templates
- token budgets
- source grounding
- observability and governance
- user-facing reliability

Observability connects those layers.

For an AI Data Platform Architect, the core question is not simply:

Did the request finish?

The better question is:

Did the system retrieve the right context, generate a timely answer, stay within token budget, avoid errors, and produce a response that was useful and grounded?

That question requires traces, metrics, dashboards, and reliability goals designed specifically for AI workloads.

This project is intentionally local and lightweight, but the architecture points toward the same capabilities needed in production systems.

It instruments the workflow.

It records AI-specific signals.

It visualizes traces.

It stores request-level metrics.

It exposes summary and event endpoints.

It defines reliability goals.

That is the foundation of AI observability.

Production Improvements

The local lab creates the foundation, but a production-grade AI observability platform would add several improvements.

Production Metrics Storage

SQLite is useful for local development, but production systems would likely use Prometheus, OTLP metrics, warehouse-backed event tables, or managed observability tooling.

The storage layer should support higher volume, retention policies, aggregation, and integration with alerting systems.

Percentile Latency

The project already tracks average latency. A production system should add p50, p95, and p99 latency aggregation.

Average latency can hide tail latency problems.

For AI systems, tail latency matters because long model calls can create poor user experiences even when average latency looks acceptable.

Real RAG Tracing

The mock workflow can be extended to trace real RAG calls from the document assistant.

That would allow the platform to observe actual retrieval latency, source count, similarity behavior, prompt construction, and answer generation.

Real Model Tracing

The observability layer can also trace real model calls from the local LLM serving layer.

That would connect model name, model latency, token counts, runtime behavior, and generation errors into the trace.

Prompt and Index Versioning

Production AI systems need to know which prompt template and retrieval index were used for a response.

Useful attributes include:

- prompt template version
- retrieval index version
- embedding model name
- embedding model version
- vector store configuration
- model name
- generation settings

These attributes make it possible to debug regressions and compare behavior across versions.

Evaluation Datasets

The local quality score is a starting point. A production system should add evaluation datasets.

Evaluation sets allow the platform to measure quality against known questions, expected sources, and expected response behavior.

This is how teams detect quality regressions when prompts, models, retrieval settings, or embeddings change.

Alerting

A production system should alert on:

- error rate spikes
- token usage spikes
- latency outliers
- retrieval degradation
- low quality scores
- model exceptions
- no-source answer rates

Alerting turns observability into operations.

Dashboard Integration

The local dashboard is useful for the lab. A production version could connect the same metrics to Grafana, Phoenix, managed observability platforms, or internal data products.

The dashboard should serve both engineering and product stakeholders.

Engineers need latency, errors, traces, and token usage.

Product and data teams may need quality trends, source usage, retrieval hit rate, and user-facing success metrics.

Distributed Trace Propagation

As the platform grows, AI requests may cross multiple services: API, worker, retrieval service, model service, evaluation service, and storage layer.

Distributed trace propagation allows one request to be followed across all of those boundaries.

That is essential for debugging production AI platforms.

Final Takeaway

The `05-ai-observability-platform` project demonstrates how to observe an AI API as a workflow, not just as an HTTP endpoint.

It instruments an AI request with OpenTelemetry spans, exports traces to Phoenix, records request-level metrics, exposes metrics endpoints, renders a local dashboard, and documents reliability goals for AI systems.

The most important lesson is that AI observability must measure both system behavior and model behavior.

Latency, errors, and uptime still matter. But for AI platforms, they are not enough.

The platform also needs visibility into retrieval, token usage, model latency, source grounding, quality signals, and AI-specific failure modes.

From an AI Data Platform Architect's perspective, this is the layer that turns AI applications into operable systems.

It gives teams the ability to trace requests, inspect bottlenecks, measure quality signals, define reliability targets, and understand whether the AI workflow is actually behaving the way the platform expects.